

Ejercicio Demostrativo

Nombre del Auxiliar

Tiempo estimado: 30 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Trabajo en Equipo	Se evidencia durante la actividad colaborativa de clasificación y selección de metodologías, donde los estudiantes discuten, justifican sus posturas y llegan a consensos basados en los factores presentados.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Compara y contrasta diferentes enfoques metodológicos para el desarrollo de software, comprendiendo su evolución histórica, sus principios fundamentales y su aplicabilidad en distintos contextos organizacionales.
Competencia Específica	El estudiante desarrolla la capacidad de identificar diferencias clave entre metodologías robustas (tradicionales) y ágiles, así como aplicar factores de selección para recomendar una metodología apropiada ante un caso práctico simulado, fundamentándose en el contenido de la unidad.

2. Palabras Clave

Metodología de Desarrollo: Conjunto de prácticas, procedimientos, reglas y documentación que guían el ciclo de vida del desarrollo de software, desde la idea inicial hasta el mantenimiento.

Metodologías Robustas (Tradicionales): Enfoques como RUP, MSF o el modelo en cascada, caracterizados por planificación detallada, fases secuenciales y énfasis en documentación extensa.

Metodologías Ágiles: Enfoques como Scrum, XP o Kanban, basados en el Manifiesto Ágil (2001), que priorizan la adaptabilidad, la colaboración con el cliente y la entrega temprana y continua de valor.

Factor de Selección: Criterio utilizado para elegir una metodología, como tamaño del equipo, incertidumbre de requisitos, criticidad del sistema, cultura organizacional o plazo disponible.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada un análisis comparativo y aplicado de las metodologías de desarrollo de software. El asistente guiará a los estudiantes a través de:

1. Una introducción conceptual clara (definiciones, historia y evolución).
2. Una actividad de clasificación práctica de metodologías (robustas vs ágiles).
3. La aplicación de una matriz de factores de selección sobre un caso simulado (desarrollo de un sistema académico universitario).
4. La justificación estructurada de una recomendación metodológica final.

La clase debe orientarse a que el estudiante relacione la teoría con escenarios reales, desarrolle criterio técnico y pueda argumentar por qué una metodología es más adecuada que otra según contexto.

4. Contenido

Descripción del ejercicio

Se plantea un escenario: una universidad desea desarrollar un sistema de gestión académica (matrícula, calificaciones, horarios). El equipo de desarrollo está compuesto por 5 personas, los requisitos son medianamente conocidos pero pueden cambiar durante el proyecto, el sistema es crítico (maneja datos sensibles) y se necesita una primera versión funcional en 4 meses. A partir de los temas vistos (introducción, historia, metodologías robustas/ágiles, factores de selección y tendencias), los estudiantes deberán recomendar una metodología, justificando cada factor evaluado.

Objetivo de la demostración

Demostrar el procedimiento para analizar y seleccionar una metodología de desarrollo de software, utilizando una matriz de factores de selección y casos de referencia (RUP, Scrum, XP), con el fin de que el estudiante comprenda cómo aplicar criterios técnicos y contextuales para tomar decisiones metodológicas fundamentadas.

Recursos necesarios

- **Software:** Navegador web, hoja de cálculo (Google Sheets / Excel) o pizarra digital colaborativa (Miro, Jamboard).
- **Plataforma / Entorno:** Google Classroom o similar para compartir materiales.
- **Herramientas técnicas:** Plantilla de matriz de selección (proporcionada por el auxiliar).

- **Archivos de apoyo:** Diapositivas resumen de las metodologías (RUP, Scrum, XP, Kanban), tabla de factores de selección.
- **Equipo necesario:** Proyector, computadora del auxiliar, pizarrón físico o digital.
- **Configuración mínima:** Acceso a internet para ejemplos y consulta del Manifiesto Ágil.

Desarrollo paso a paso

Paso 1

Acción inicial: Introducción al contexto y activación de conocimientos previos.

Objetivo: Establecer por qué las metodologías son importantes y qué problema resuelven.

Qué observar: El auxiliar proyecta las diapositivas iniciales (historia: crisis del software, años 60-70 -> enfoques robustos; años 90 -> necesidad de cambio -> Manifiesto Ágil 2001). Luego pregunta: "¿Por qué hay tantas metodologías distintas?" y guía una lluvia de ideas breve (3-5 min).

Paso 2

Acción: Exposición diferenciada de metodologías robustas vs ágiles.

Explicación de criterios: Se presentan dos grandes grupos:

- **Robustas:** Planificación predictiva, fases rígidas, documentación extensa, control de cambios riguroso. Ejemplo: RUP.
- **Ágiles:** Adaptativa, ciclos cortos (sprints), colaboración con cliente, software funcionando sobre documentación. Ejemplo: Scrum, XP.

Validación importante: El auxiliar muestra una tabla comparativa (ver Tabla 1) y pide a los estudiantes que identifiquen qué metodología usarían para un proyecto muy pequeño y uno muy grande. Se discuten respuestas.

Paso 3

Ejecución principal: Aplicación de factores de selección al caso simulado.

Detalle: Se entrega a los estudiantes una matriz con los siguientes factores y se completa paso a paso con apoyo del auxiliar:

Factor	Valor / Condición en el caso	Peso (1-3)	Robustas	Ágiles	Justificación breve
Tamaño del equipo	5 personas	2	Media	Alta	Equipos pequeños favorecen agilidad
Incertidumbre de requisitos	Media (pueden cambiar)	3	Baja	Alta	Métodos ágiles manejan mejor el cambio
Criticidad del sistema	Alta (datos sensibles)	3	Alta	Media	Robustas tienen más controles formales
Plazo disponible	4 meses (corto)	2	Baja	Alta	Ágiles entregan rápido
Cultura organizacional	Academia, abierta al cambio	2	Media	Media	Depende del equipo real

Interpretación de resultado parcial: Se suman puntajes. En este caso, ágiles obtienen mayor puntaje (aprox 10 vs 7 en robustas), pero la criticidad obliga a considerar controles adicionales (ej. XP con prácticas de calidad extrema).

Paso 4

Cierre técnico (recomendación final y tendencias): El auxiliar guía la construcción de una recomendación estructurada y menciona tendencias actuales (DevOps, híbridos como Scrumban, metodologías en la nube).

Estructura de recomendación:

"Con base en los factores analizados, se recomienda **Scrum** con prácticas complementarias de XP (programación en pares, integración continua) porque:

- Equipo pequeño y plazo corto → Scrum permite sprints de 2 semanas.
 - Criticidad alta → XP refuerza calidad técnica.
 - Requisitos cambiantes → product backlog priorizable.
- Además, se sugiere automatizar pruebas (tendencia DevOps) para mejorar confiabilidad."

Confirmación de aprendizaje: Los estudiantes redactan individualmente su recomendación final (5 min) y dos voluntarios la comparten. El auxiliar corrige o refuerza conceptos.

Observaciones importantes

- **Error común:** Creer que "ágil" significa "sin planificación" o "sin documentación".
Aclarar que la documentación es *justa y necesaria*.
- **Buena práctica:** Usar el Manifiesto Ágil (2001) como documento base para discutir principios, no solo prácticas.
- **Advertencia:** No seleccionar metodología por moda; siempre basarse en factores concretos del proyecto y equipo.
- **Recomendación:** Invitar a los estudiantes a aplicar la misma matriz a sus propios proyectos (personales o de otros cursos).

Tabla 1 – Comparativa de Metodologías Representativas

Elemento	RUP (Robusta)	Scrum (Ágil)	XP (Ágil)
Ciclo de vida	Iterativo incremental con fases (Inicio, Elaboración, Construcción, Transición)	Sprints de 1-4 semanas	Iteraciones de 1-3 semanas
Roles principales	Analista, arquitecto, desarrollador, tester	Product Owner, Scrum Master, Equipo	Cliente, programador, tester, tracker
Documentación	Extensa, artefactos por fase	Mínima (backlog, definición de done)	Código como documentación principal

Métricas clave	Hitos de fase, hitos de arquitectura	Velocidad, burndown chart	Pruebas unitarias, cobertura de código
Cuándo usarla	Sistemas grandes, críticos, equipos grandes distribuidos	Requisitos cambiantes, equipos multidisciplinarios pequeños/medios	Calidad crítica, equipos pequeños, requisitos que cambian rápido

Resultado esperado

Al finalizar el ejercicio, cada estudiante debe ser capaz de producir:

- Una **matriz de factores de selección** completada para el caso simulado.
- Una **recomendación metodológica escrita** (1-2 párrafos) que justifique su elección usando al menos 3 factores vistos en clase.
- Identificar correctamente si una metodología dada (por el auxiliar) pertenece al grupo robusto o ágil, mencionando al menos dos características distintivas.

Aplicación práctica

Este ejercicio conecta directamente con:

- La **vida profesional temprana** de un ingeniero: al integrarse a un equipo, deberá comprender y adaptarse a la metodología vigente.
- La **toma de decisiones técnicas**: saber argumentar por qué cierta metodología es mejor para cierto proyecto (útil en entrevistas, análisis de casos, planificación de proyectos de software).
- Otros cursos del área como *Administración de Proyectos*, *Ingeniería de Requisitos* y *Calidad de Software*.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- Algunos estudiantes tienden a memorizar nombres de metodologías sin comprender sus principios subyacentes.
- Confusión entre "Scrum" y "XP": los roles y prácticas específicas se mezclan con facilidad.
- Dificultad para aplicar factores abstractos (como "cultura organizacional") sin ejemplos muy concretos.

Recomendaciones para futuras sesiones:

- Empezar con una **actividad de contraste rápido**: mostrar dos viñetas de proyectos (ej. un satélite vs una aplicación móvil) y preguntar qué metodología usarían *intuitivamente* antes de dar teoría.
- Usar **analogías**: metodologías robustas como construir un puente (todo planificado), ágiles como preparar una comida con ingredientes que cambian (ajuste continuo).
- Proveer una **hoja de respuestas parcial** después del paso 3 para que los estudiantes que van más lento puedan nivelarse.
- Asignar tarea corta: buscar un caso real en internet de éxito/fracaso metodológico y compartirlo en el foro del curso.

Qué puntos consideras fue de mayor dificultad para el estudiante:

- Comprender que **robusto no es "malo"** y ágil no es "siempre mejor". La madurez para elegir según contexto cuesta al inicio.
- Diferenciar **Scrum** (gestión) de **XP** (prácticas de ingeniería). Suelen confundirse porque se usan juntos frecuentemente.
- La dimensión histórica: por qué surgieron las ágiles como respuesta, no como capricho. Sin contexto, parece un moda.

6. Referencias

- Beck, K. et al. (2001). *Manifiesto Ágil para el Desarrollo de Software*. Recuperado de <https://agilemanifesto.org/iso/es/>
- Pressman, R. (2021). *Ingeniería del Software: Un Enfoque Práctico* (9ª ed.). McGraw-Hill.
- Schwaber, K. & Sutherland, J. (2020). *La Guía Scrum*. Recuperado de <https://scrumguides.org/>
- Sommerville, I. (2019). *Ingeniería de Software* (10ª ed.). Pearson.
- Project Management Institute. (2021). *PMBOK Guide* (7ª ed.) – sección sobre enfoques predictivos vs adaptativos.